

L02. Markov Decision Processes and Optimal Control

*Lecturer: Munther Dahleh**Scribe: Jinhee Won*

1 Introduction

Our study of reinforcement learning is a study of dynamic processes. In supervised or unsupervised learning, we generally consider data at only one point in time. For example, we might observe an input x_i together with a corresponding outcome y_i , as in a regression, decision-tree, or clustering problem. In reinforcement learning, however, we must model how the problem changes over time. This requires notation and modeling choices that accurately represent the dynamics of the problem.

Consider COVID-19 as an example. A classical regression problem might use a patient's symptoms to predict whether the patient has COVID-19. In this lecture, however, we are interested in a dynamic problem in which COVID-19 spreads through a population.

A simple epidemiological model might define the state using the numbers of infected and uninfected people. This immediately raises a complication: asymptomatic individuals may be infected even though their condition is not directly observable. We may therefore observe only a measurement of the state rather than the true state itself. Consequently, how we measure the system and how we define its state are fundamental parts of the problem formulation.

2 Markov Processes

We continue from the state-space formulation introduced in the previous lecture.

2.1 Simple Linear Transition

Recall that the state at time t is denoted by

$$s_t \in S.$$

The state evolves according to the dynamics

$$s_{t+1} = f_t(s_t, w_t),$$

where w_t represents the uncertainty in the system and has an associated probability distribution $\mathbb{P}(\cdot)$.

The Markov property states that, conditional on the current state s_t , the future is independent of the past. More precisely,

$$\mathbb{P}(s_{t+1} \mid s_t, s_{t-1}, \dots, s_0) = \mathbb{P}(s_{t+1} \mid s_t).$$

Consider the simple linear model

$$s_{t+1} = \frac{1}{2}s_t + w_t,$$

where the disturbances w_t are independent and identically distributed.

An advantage of the linear representation is that we can obtain a closed-form expression for the state. Repeatedly substituting the transition equation gives

$$s_t = \left(\frac{1}{2}\right)^t s_0 + \sum_{\tau=0}^{t-1} \left(\frac{1}{2}\right)^{t-\tau-1} w_\tau.$$

This expression is a convolution of the sequence of disturbances with the system's response. It shows explicitly how the state at time t depends on the initial state and on all previous disturbances.

2.2 Simple Queuing Transition

As another example, let s_t denote the number of jobs in a queue and let w_t denote the number of new arrivals. If one job can be served during each period, the queue evolves according to

$$s_{t+1} = \max\{0, s_t + w_t - 1\}.$$

The maximum ensures that the queue length cannot become negative.

If the distribution of the arrivals depends on the current queue length, then the disturbance is state-dependent. We can represent it as

$$w_t = g(s_t, \bar{w}_t),$$

where $\bar{w}_t$ is an independent source of randomness. The queue dynamics can then be rewritten as

$$s_{t+1} = \max\{0, s_t + g(s_t, \bar{w}_t) - 1\}.$$

Thus, state-dependent uncertainty can still be expressed as a function of the current state and an independent random disturbance.

2.3 Probabilistic Representation

So far, we have represented a Markov process using a dynamics function and, in some cases, a closed-form solution. We show that we can also represent the process directly in terms of transition probabilities.

Let S be a finite state space with $|S| = n$. Suppose the dynamics are

$$s_{t+1} = f(s_t, w_t).$$

Rather than specifying the function f explicitly, we can describe how the state changes by giving the probability of moving from each current state s to each possible next state s' . Define the transition probability

$$P(s'; s) = \mathbb{P}(s_{t+1} = s' \mid s_t = s).$$

Using the dynamics function, this can also be written as

$$P(s'; s) = \mathbb{P}(f(s, w_t) = s').$$

Thus, a dynamics model naturally determines a probabilistic transition model.

The reverse direction is less immediate. At first glance, the relationship may appear to be one-to-many rather than an equivalence between the two representations. However, we can show that a transition-probability model can also be expressed using a dynamics function driven by a random variable.

Given the transition probabilities $P(s'; s)$, we can construct a dynamics function driven by a random variable. For example, let

$$w_t \sim \text{Uniform}[0, 1].$$

For each current state s , divide the interval $[0, 1]$ into regions whose lengths are the probabilities $P(s'; s)$. The function $f(s, w_t)$ selects the next state according to the region containing the sampled value of w_t . Therefore, the transition probabilities can also be represented through dynamics of the form

$$s_{t+1} = f(s_t, w_t).$$

Aside

I think it is helpful to think of this construction as a decision tree. From the current state, each possible next state is a branch with a particular probability. Sampling a uniform random variable selects one of those branches, allowing us to rewrite the probabilistic model in the form $s_{t+1} = f(s_t, w_t)$.

Constructing this function explicitly is usually unnecessary. The main point is the conceptual equivalence between the dynamics representation and the transition-probability representation.

3 Markov Decision Processes

A Markov decision process contains everything discussed for a Markov process, but now includes a decision. We represent the decision at time t by an action

$$a_t \in A_t(s_t),$$

where $A_t(s_t)$ is the set of actions available in state s_t .

A Markov decision process consists of the following components:

1. a state

$$s_t \in S;$$

2. dynamics

$$s_{t+1} = f_t(s_t, a_t, w_t);$$

3. a stage cost

$$c_t(s_t, a_t, w_t);$$

4. the action constraint

$$a_t \in A_t(s_t);$$

5. and a model of uncertainty for w_t , such as

$$\mathbb{P}(\cdot \mid s_t, a_t).$$

In order to keep the Markovian property, note that the action

$$a_t = \mu_t(s_t),$$

may only be dependent on the current state.

3.1 Example

For example, suppose the state in a stock problem initially consists only of the current price p_t . However, if we want to know whether or not we should buy/hold/sell the stock, we likely need more information than just that. For instance, we may want our action to depend on both the current price and the previous price,

$$a_t = \mu_t(p_t, p_{t-1}).$$

That means that p_t alone is not a sufficient state, and we must instead augment the state by defining

$$s_t = \begin{pmatrix} p_t \\ p_{t-1} \end{pmatrix}.$$

Aside

In real applications, the chosen state may not be perfectly Markovian. A common approach is to first develop the theory under the Markov assumption and then study whether the selected state behaves sufficiently like a Markov state in practice. However, it is important that we assume this Markovian property in order to keep some of our future theoretically optimal proofs.

3.2 Probabilistic Representation, Extended

As with an uncontrolled Markov process, an MDP can be represented either by its dynamics or by transition probabilities. The dynamics

$$s_{t+1} = f_t(s_t, a_t, w_t)$$

can also be written as

$$p_t(s'; s, a) = \mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a).$$

3.3 Revisiting the Queuing Problem

We can now formulate the queuing problem a little more concretely. A service facility receives customers at random times, and the manager controls the service rate by allocating resources. For example, the manager may add or remove servers, allocate additional compute or labor, or prioritize particular customer classes.

The objective is to balance two competing costs: the cost of allowing customers to wait in a long queue and the cost of providing a higher service rate.

Let n_t denote the number of customers in the queue at time t . The queue evolves according to

$$n_{t+1} = \max\{0, n_t + A_t - S_t\},$$

where

$$A_t \sim \text{Poisson}(\lambda)$$

is the number of arrivals during the interval $[t, t + 1]$, and

$$S_t \sim \text{Poisson}(\mu_t)$$

is the number of completed services during that interval. The arrival rate λ is fixed, while the service rate μ_t is selected by the manager and serves as the action.

We now ask whether the current queue length n_t is a sufficient state.

From an individual customer's perspective, it may initially seem that it is not. For example, a particular customer may appear to be three periods away from completion at time t and two periods away from completion at time $t + 1$. The model does not track that individual customer's remaining service time. In other words, it is not designed to describe the system at the level of individual customers.

However, we do not need to track the identity of each customer or the amount of service time each customer has already received. The Poisson-process model has independent increments, while the associated exponential service-time model is *memoryless*. Therefore, conditional on the current queue length and the selected service rate, the past does not provide additional information about the future evolution of the queue.

Aside

The memoryless assumption makes the state representation much cleaner, but it may not be realistic for every queue. For example, delays on the Red Line may be persistent: if a train is already late, it may be more likely to remain late. In such a setting, the current queue length alone may not contain enough information, and the state may need to include additional variables such as elapsed waiting or service times.

3.4 Autoregressive Models

Autoregressive models provide another example of how a process that depends on the past can be written in state-space form. Consider the model

$$y_t = \sum_{i=1}^p d_i y_{t-i} + b a_{t-1} + \epsilon_{t-1},$$

where a_t is an input and ϵ_t is a random disturbance.

The value y_t alone is not a sufficient state because the next output depends on the previous p outputs. We can instead collect those outputs into the state

$$S_t = \begin{bmatrix} y_{t-p+1} \\ y_{t-p+2} \\ \vdots \\ y_t \end{bmatrix}.$$

The state then evolves according to

$$S_{t+1} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 1 \\ d_p & d_{p-1} & \cdots & d_2 & d_1 \end{bmatrix} S_t + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ b \end{bmatrix} a_t + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \epsilon_t.$$

By including the required history in S_t , the higher-order autoregressive model becomes a first-order state-space model.

We can apply the same idea to an autoregressive moving-average model with inputs

$$y_t = \sum_{i=1}^p d_i y_{t-i} + \sum_{i=1}^{m_1} b_i a_{t-i} + \epsilon_t + \sum_{i=1}^{m_2} c_i \epsilon_{t-i}.$$

We leave the construction of the state space as an exercise to the reader.

4 Specifying the Objective

With our terminology and notation to describe a dynamic process, we can now formalize our objective.

Recall from probability that, for any function g of the next state,

$$\mathbb{E}[g(s_{t+1}) \mid s_t = s, a_t = a] = \sum_{s' \in S} g(s') p_t(s'; s, a).$$

If $g(s')$ represents the future cost (or reward) associated with reaching state s' , this expression gives its expected value under the transition probabilities! This allows us to evaluate the expected future consequences of each action.

We define a policy π as a sequence of decision rules that determine which action is selected in each state. Given an initial state s_0 , we can then define the finite-horizon expected cost

$$V_N^\pi(s_0) = \mathbb{E}^\pi \left[c_N(s_N) + \sum_{t=0}^{N-1} c_t(s_t, \mu_t(s_t), w_t) \right].$$

Here, c_t is the stage cost incurred at time t , while $c_N(s_N)$ is the terminal cost at the end of the horizon.

Our goal then is to find a policy that minimizes the expected total cost:

$$\pi^* \in \arg \min_{\pi} V_N^\pi(s_0).$$

Aside

Note that this "cost" terminology is often used in the control communities, but also often referred to as reward in the RL community. This course will often switch between the two, but note that $c_t = -r_t$, and the optimization becomes a maximization problem rather than a minimization problem.